

애니메이션 최적화 : 멀티스레딩

영웅호걸 4차 코드리뷰

- 윤지원 -

문제 인식

Visual Studio 프로파일링 기능으로
Bone 행렬 연산이 CPU 부하의 큰
비중을 차지하는 것을 발견

Client(PID: 31460)	1549(100.00%)	0(0.00%)
[외부 호출] ntdll!0x00007f6bbd4ad6c	1549(100.00%)	119(7.68%)
__scrt_common_main_seh	1365(88.12%)	0(0.00%)
__scrt_common_main	1365(88.12%)	0(0.00%)
wWinMainCRTStartup	1365(88.12%)	0(0.00%)
wWinMain	1365(88.12%)	0(0.00%)
invoke_main	1365(88.12%)	0(0.00%)
Client::CMainApp::Update	719(46.42%)	0(0.00%)
Engine::CGameInstance::Update_Engine	661(42.67%)	0(0.00%)
Client::CMainApp::Render	646(41.70%)	0(0.00%)
Engine::CRenderer::Draw	619(39.96%)	0(0.00%)
Engine::CGameInstance::Draw	619(39.96%)	0(0.00%)
Engine::CObject_Manager::Update	429(27.70%)	0(0.00%)
Engine::CLayer::Update	428(27.63%)	0(0.00%)
Engine::CModel::Play_Animation	392(25.31%)	0(0.00%)
Engine::CModel::Apply_Animation	377(24.34%)	1(0.06%)
Engine::CRenderer::Ready_Shadow	314(20.27%)	1(0.06%)
Engine::CCollision_Manager::CollisionUpdate	203(13.11%)	3(0.19%)
Engine::CRenderer::Render_NonBlend	167(10.78%)	0(0.00%)
?ApplyPassBlock@CEffect@D3DX11Effects@@@IEAAXPEAUSPassBlo...	146(9.43%)	0(0.00%)
?Apply@SPassBlock@D3DX11Effects@@@UEAAJIPEAUID3D11Devic...	146(9.43%)	0(0.00%)
Engine::CShader::Begin	140(9.04%)	2(0.13%)
Engine::CMesh::Update_BoneMatrix	140(9.04%)	16(1.03%)
?ApplyShaderBlock@CEffect@D3DX11Effects@@@IEAAXPEAUSShad...	128(8.26%)	1(0.06%)
Engine::CChannel::Update_TransformationMatrix	116(7.49%)	16(1.03%)
Engine::CAnimation::Update_TransformationMatrices	114(7.36%)	0(0.00%)
DirectX::XMMATRIX::operator*	87(5.62%)	14(0.90%)

Debug 모드에서 37 ~ 38FPS

시도 1 : 멀티스레딩을 활용하여 Bone 행렬 연산 병렬화

```
void CMesh::Update_BoneMatrix(const vector<class CBone*>& Bones)
{
    ZeroMemory(m_BoneMatrices, sizeof(_float4x4) * g_iMaxNumBones);

    auto Func = [&](size_t iStart, size_t iEnd)
    {
        for (size_t i = iStart; i < iEnd; ++i)
        {
            XMStoreFloat4x4(&m_BoneMatrices[i],
                XMLoadFloat4x4(&m_OffsetMatrices[i]) *
                Bones[m_Bones[i]]->Get_CombinedTransformationMatrix());
        }
    };

    size_t iHalf = m_iNumBones / 2;

    thread t1(Func, 0, iHalf);
    thread t2(Func, iHalf, m_iNumBones);

    t1.join();
    t2.join();
}
```

결과

기대한 성능이 나오지 않았으며, 오히려 프레임이 급격하게 떨어짐 => 3 ~ 4 FPS

시도 1 실패 원인 분석

원인 1 : 너무 작은 작업 단위 - **False Sharing**

사용하는 리소스 중 가장 많이 사용하는 **bone**의 개수가 200개 이내

심지어 **Mesh**에 영향을 주는 **bone**의 개수는 이보다 더 적었을 것.

한 메쉬 내의 **bone** 행렬 연산 수 자체가 적은데, 이를 병렬화 하려고 하니, **Cache** 무효화로 인해 싱글스레드보다 성능이 떨어짐

시도 2 : 작업 단위를 키워 Model 묶음 별 병렬 처리

```
class ENGINE_DLL CGameObject abstract : public CBase
{
public:
    typedef struct tagGameObjectDesc : public CTransform::TRANSFORM_DESC
    {
        _tchar m_szTagBuffer[MAX_PATH] = TEXT("");
        const _tchar* pGameObjectTag;
    } GAMEOBJECT_DESC;

private:
    static _uint g_guid;
    _uint m_guid;

protected:
    CGameObject();
    CGameObject(ComPtr<ID3D11Device> pDevice, ComPtr<ID3D11DeviceContext> pContext);
    CGameObject(const CGameObject& Prototype);
public:
    virtual ~CGameObject() = default;

public:
    _uint Get_GUID() const { return m_guid; }
    const _uint& GetMyGUID(void) const { return m_guid; }
    virtual HRESULT Initialize_Prototype();
    virtual HRESULT Initialize(void* pArg);
    virtual void Priority_Update(_float fTimeDelta);
    virtual void Update(_float fTimeDelta);
    virtual void Update_Animation(_float fTimeDelta) {}
    virtual void Last_Update(_float fTimeDelta);
    virtual HRESULT Render();
    virtual HRESULT Render_Shadow() { return S_OK; }
    virtual HRESULT Render_Outline() { return S_OK; }
    virtual HRESULT Render_GlowMask();
    virtual _vector Get_Position();
```

Animation 처리 단계를 분리

Object manager에서 Animation을 사용하는 모델을 묶어서 멀티스레딩을 수행

```
void CObject_Manager::Update(_float fTimeDelta)
{
    Update_SlowTime(fTimeDelta);
    _float fScale = Get_CurrentTimeScale();

    // Phase 1: 게임 로직 (직렬) - FSM, 상태 변경 등
    for (size_t i = 0; i < m_iNumLevels; i++)
        for (auto& Pair : m_pLayers[i])
            Pair.second->Update(fTimeDelta, fScale);

    // Phase 2: 애니메이션 (병렬) - 모델 별 독립 실행
    vector<shared_ptr<CLoadTask>> tasks;
    for (size_t i = 0; i < m_iNumLevels; i++)
        for (auto& Pair : m_pLayers[i])
            Pair.second->Collect_AnimationTasks(fTimeDelta, fScale, tasks);

    m_pGameInstance->Submit_And_Wait(tasks);
}
```

결과

37->45FPS로 늘어남!!

그러나, 의존성 문제를 조심해야 함